

Measure and Conquer: Analysis of a Branch-and-Reduce Algorithm for the Maximum Bounded-degree-1 Set Problem*

Maw-Shang Chang[†]

Department of Computer Science and Information Engineering
HungKuang University, Taichung 43302, Taiwan
`mschang@sunrise.hk.edu.tw`

Ling-Ju Hung, and Ping-Chen Su

Department of Computer Science and Information Engineering
National Chung Cheng University, Chiayi 62102, Taiwan
`{hunglc, spc98m}@cs.ccu.edu.tw`

Abstract

Recently Measure and Conquer is used to obtain tighter bounds of running time of branch-and-reduce algorithms. A graph is bounded-degree-1 if its maximum degree is less than or equal to one. The maximum bounded-degree-1 set problem is to find a vertex subset of maximum size that induces a bounded-degree-1 subgraph. The problem that to find a maximum bounded-degree-1 set in a given graph is NP-hard. Chang et al. gave an $O^(1.5171^n)$ -time exact algorithm to solve it where n is the number of vertices in the input graph [7]. In this paper, we modify this algorithm and analyze the new algorithm by the measure and conquer approach. We show that the running time of the new algorithm is $O^*(1.4834^n)$.*

1 Introduction

The technique *Branch and Reduce* is one of the major techniques used to design exact algorithms. It is a recursive procedure consisting of *reduction rules* and *branching rules*. The reduction rules are used to reduce the problem size, and the branching rules are used to branch the original problem into subproblems having smaller input sizes. Branch-and-reduce algorithms always find

optimal solutions. Search trees are often used to illustrate the execution of a branching algorithm. Let $T(n)$ be the maximum number of leaves in any search tree where n is the input size. We analyze each branching rule and use the worst-case time complexity over all branching rules as an upper bound of the running time. Let b be any branching rule. When rule b is applied, the current instance is branched into $r \geq 2$ instances of size at most $n - t_1, n - t_2, \dots, n - t_r$. Note that $n \geq t_i$ and $t_i \in \mathbb{R}^+$ for $i = 1, 2, \dots, r$. We call $\mathbf{b} = (t_1, t_2, \dots, t_r)$ the *branching vector* of branching rule b . This can be formulated in a linear recurrence

$$T(n) \leq T(n - t_1) + T(n - t_2) + \dots + T(n - t_r).$$

For functions f and g we write $f(n) = O^*(g(n))$ if $f(n) = O(g(n) \text{poly}(n))$, where $\text{poly}(n)$ is a polynomial. Then $T(n) = O^*(\tau^n)$ where τ is the unique positive real root of

$$x^n - x^{n-t_1} - x^{n-t_2} - \dots - x^{n-t_r} = 0.$$

We call τ the *branching number*¹ of branching rule b [14]. If a branch-and-reduce algorithm has several branching rules, then the running time of it is $O^*(\alpha^n)$ where α is the maximum branching number among all branching rules.

Recently *measure and conquer* is used to obtain tighter bounds on running time of many branch-and-reduce algorithms [8, 4, 11, 9, 12, 13, 6, 14].

*This research is partially supported by the National Science Council of Taiwan under grants NSC 98-2221-E-194-026-MY3 and NSC 99-2221-E-194-017-MY3.

[†]This work was partially done while this author was with Department of Computer Science and Information Engineering, National Chung Cheng University.

¹To compute the branching number of a branching vector, please visit the following webpage: <http://ufo.cs.ccu.edu.tw/resources.htm>

Traditionally the size of a graph problem is measured by the number n of vertices in the input graph. In measure and conquer analysis, each vertex is assigned a weight in the range $[0, 1]$ and the size of the problem is measured by the total weight w of all vertices. Usually the weight of a vertex depends on its degree in the graph and the higher its degree the heavier its weight. Hence the weight of a vertex is reduced while its degree is reduced during the execution of the algorithm. The key issue in measure and conquer analysis is how to assign weights to vertices to obtain a tighter bound on the running time of the algorithm. The problem of assigning weights to vertices to obtain a tighter bound on the running time was formulated as a quasiconvex program [9]. Eppstein showed how to solve the quasiconvex program by a greedy algorithm [8, 9]. We implemented this algorithm and used it to analyze the running time of our branch-and-reduce algorithm for the MAX-1-BDS problem defined as follows.

Let $G = (V, E)$ be a simple undirected graph. Use $G[X]$ to denote the graph induced by X in G . For a vertex v in G , use $N(v)$ for the set of vertices adjacent to v and let $N[v] = N(v) \cup \{v\}$. We call $|N(v)|$ the *degree* of v , denoted by $d_G(v)$. Let $\Delta(G)$ denote the maximum degree of G . We call a graph G *BOUNDED-DEGREE- d* iff $\Delta(G) \leq d$. A vertex subset S of G is called a *BOUNDED-DEGREE- d Set* of G if $\Delta(G[S]) \leq d$.

Maximum Bounded-Degree- d Set
(MAX- d -BDS)

Input: A graph $G = (V, E)$.

Output: A vertex set $S \subseteq V$ of maximum cardinality such that $G[S]$ is BOUNDED-DEGREE- d .

A k -plex of a graph $G = (V, E)$ is a vertex set $S \subseteq V$ such that every vertex in $G[S]$ has degree at least $|S| - k$. By definition, a k -plex in G is a $(k - 1)$ -BDS in the complement graph of G , say $\bar{G}$. Thus a graph G with n vertices has a k -plex of size s iff $\bar{G}$ has a $(k - 1)$ -BDS with s vertices. The model k -plex is defined as a degree based relaxed model for finding cohesive subgroups in social networks [19]. Notice that a 1-plex is a clique. The *maximum k -plex problem* in a given graph G is to find a k -plex in G with maximum cardinality. Balasundaram *et al.* proved that the maximum k -plex problem is NP-complete [2]. They gave a 0/1 integer program for the problem [1, 2]. It is known that the maximum k -plex problem is W[1]-hard with respect to the parameter s being the size of k -plexes [15]. Some branch-and-bound algorithms were given for solving the maximum k -plex problem based on different upper bounds

and lower bounds found by some heuristics algorithms [20, 16]. A series of fixed algorithms were developed for solving the MAX- d -BDS problem by taking the size of vertices being deleted as the input parameter [18, 15, 17, 10]. When parameterized by treewidth the MAX-1-BDS problem is W[1]-hard [5]. In [3], Balasundaram *et al.* referred the MAX- d -BDS problem as the problem of finding maximum-cardinality co- $(d + 1)$ -plexes. Chang *et al.* gave an $O^*(1.5171^n)$ -time exact algorithm to solve the MAX-1-BDS problem where n is the number of vertices in the input graph [7]. In this paper, we modify the algorithm in [7] and analyze the new algorithm by the measure and conquer approach. We show that the running time of the new algorithm is $O^*(1.4834^n)$.

We close this section with some notation. Given a graph $G = (V, E)$, we use n to denote the number of vertices in G . Two vertices u and v are called *true twins* in G if $N[u] = N[v]$. Two vertices u and v are called *false twins* in G if $N(u) = N(v)$. A vertex v is called an *isolated vertex* if $|N(v)| = 0$ and is called a *degree-one vertex* if $|N(v)| = 1$. For an edge $(u, v) \in E$, define the degree of (u, v) to be $\deg(u, v) = |N(u) \cup N(v) \setminus \{u, v\}|$. An edge (u, v) is called an *isolated edge* if $\deg(u, v) = 0$ and is called a *degree-one edge* if $\deg(u, v) = 1$. We use $G \setminus X$ to denote $G[V \setminus X]$. Let $W \subseteq E$ be a subset of edge set, use $V(W) = \cup_{(u,v) \in W} \{u, v\}$. For a vertex subset $U \subseteq V$, let $N(U) = \cup_{u \in U} N(u) \setminus U$ and $N[U] = N(U) \cup U$. We use $N^i(v)$ to denote the set of vertices at distance i from v . A vertex u has *private neighbors* with respect to vertex v if $N(u) \setminus N[v] \neq \emptyset$ and p is a private neighbor of u with respect to v if $p \in N(u) \setminus N[v]$.

2 An exact algorithm for the MAX-1-BDS problem

In this section, we present a branch-and-reduce algorithm for the MAX-1-BDS problem and analyze the running time of the algorithm using the number n of vertices in the input graph as the size of the input.

When the algorithm puts a vertex v into the solution set, we say that it *selects* v . When the algorithm decides that a vertex v is not in the solution set, we say that it *discards* v . Before explaining the algorithm in detail, we describe the main rules that the algorithm will apply. Some proofs are omitted due to the space limit. For simplicity, we denote the size of a maximum bounded-degree- d set of an undirected graph G by $\beta_d(G)$.

Before we present the algorithm, we define a rule called **Standard Branching Rule**: When the algorithm applies this rule, it selects a vertex v of degree d and recursively solves $d + 2$ MAX-1-BDS problems on graphs $G_i = G \setminus U_i$, respectively, where $1 \leq i \leq d + 2$, $N(v) = \{u_1, u_2, \dots, u_d\}$, $U_1 = \{v\}$, $U_2 = N[v]$, and for $i > 2$, $U_i = N[\{v, u_{i-2}\}]$. And then $\beta_1(G) = \max\{\beta_1(G_1), 1 + \beta_1(G_2), \max_{3 \leq i \leq d+2} (2 + \beta_1(G_i))\}$. In the first branch of standard branching rule, it discards v . In the second branch, it selects v and discards $N(v)$. In each of the other branches, it selects v and a neighbor u and discards $N(\{v, u\})$.

The description of the algorithm consists of a sequence of cases and subcases. To avoid a confusing nesting of if-then-else statements let us use the following convention: The first case which applies is used in the algorithm. Thus, inside a given case, the hypotheses of all previous cases are assumed to be false. Algorithm $\beta_1(G)$ computes for a given graph $G = (V, E)$ the maximum size of a vertex set S inducing a bounded-degree-1 subgraph.

1. **Maximum Degree-Two Rule.** All vertices are of degree at most two. In this case we solve the problem in polynomial time.
2. **Isolated-Vertex Rule.** There is an isolated vertex and let X be the set of all isolated vertices. Then $\beta_1(G) = |X| + \beta_1(G \setminus X)$.
3. **Isolated-Edge Rule.** There is an isolated edge and let X be the set of all isolated edges. Then $\beta_1(G) = 2 \cdot |X| + \beta_1(G \setminus V(X))$.
4. **Degree-One Edge Rule.** There is an edge (u, v) of degree one and let $X = N[\{u, v\}]$. Then $\beta_1(G) = 2 + \beta_1(G \setminus X)$.
5. **Degree-One Vertex Rule 1.** There is a vertex v having at least two degree-one neighbors and let X be the set of all degree-one neighbors of v . Then $\beta_1(G) = |X| + \beta_1(G \setminus (X \cup \{v\}))$.
6. **True-Twin Rule 1.** There are two vertices v and z such that $N[z] = N[v]$, i.e., v and z are true twins and $d_G(v) = d_G(z) = 3$. The algorithm selects $\{v, z\}$ and discards $N(v) \setminus \{z\}$. Hence $\beta_1(G) = 2 + \beta_1(G \setminus N[v])$.
7. **True-Twin Rule 2.** There are two vertices v and z such that $N[z] = N[v]$, i.e., v and z are true twins as Fig. 1(a). Since Degree-One Edge Rule and True-Twin Rule 1 are not applicable, the degrees of v and z are at least

four. The algorithm either discards $\{v, z\}$ or selects $\{v, z\}$ and discards $N[v] \setminus \{v, z\}$. Hence $\beta_1(G) = \max\{\beta_1(G \setminus \{v, z\}), 2 + \beta_1(G \setminus N[v])\}$. The worst branching vector is $(2, 5)$ and its branching number is 1.2366.

8. **False-Twin Rule.** There exist two vertices u and v that are false twins in G as shown in Fig. 1(b). The algorithm either discards both u and v or selects both u and v . That is $\beta_1(G) = \max\{\beta_1(G \setminus \{u, v\}), 2 + \beta_1(G \setminus (N[v] \cup \{u\}))\}$. Since $d_G(u) = d_G(v) \geq 2$, the worst branching vector is $(2, 4)$ and its branching number is 1.2721.
9. **Degree-One Vertex Rule 2.** There is a vertex v having a degree-one neighbor z as Fig. 1(c). Since Degree-One Vertex Rule 1 and Degree-One Edge Rule are not applicable, the degree of v is at least three and the neighbors of v other than z are of degree at least two. Otherwise (v, z) is an edge of degree at most one. In this case the algorithm either discards v and selects z or selects $\{v, z\}$ and discards $N[v] \setminus \{v, z\}$, i.e., $\beta_1(G) = \max\{1 + \beta_1(G \setminus \{v, z\}), 2 + \beta_1(G \setminus N[v])\}$. The worst branching vector is $(2, 4)$ and its branching number is 1.2721.
10. **Domination Rule 1.** Let v be a vertex of G and u be a neighbor of v satisfying that $N[u] \subset N[v]$ as Fig. 1(d). The degree of v is at least 3. The algorithm either discards v or selects $\{v, u\}$ and discards $N[v] \setminus \{v, u\}$. Hence $\beta_1(G) = \max\{\beta_1(G \setminus \{v\}), 2 + \beta_1(G \setminus N[v])\}$. The worst branching vector is $(1, 4)$ and its branching number is 1.3803.
11. **Degree-Two Vertex Rule 1.** There is a degree-two vertex z satisfying the condition that the neighbor v with larger degree between the two neighbors of z is of degree at least three. Let x be the neighbor of z other than v . Since Domination Rule is not applicable, x and v are not adjacent and $|N[u] \setminus N[v]| \geq 1$ for any $u \in N(v)$. The degree of x is two as Fig. 1(e). The algorithm either selects both x and z or selects v and z or selects x, y, v and one of v 's neighbor p other than z . That is $\beta_1(G) = \max\{2 + \beta_1(G \setminus (N[x] \cup N[z])), 2 + \beta_1(G \setminus (N[v] \cup N[z])), 4 + \max_{U \in \mathbb{U}} (\beta_1(G \setminus (N[U] \cup N[\{x, y\}]))\}$ where $\mathbb{U} = \{\{v, p\} \mid p \in N(v), p \neq z\}$. The worst case happens when degree of v is three. The branching vector is $(4, 5, 6, 6)$ and its branching number is 1.3086.

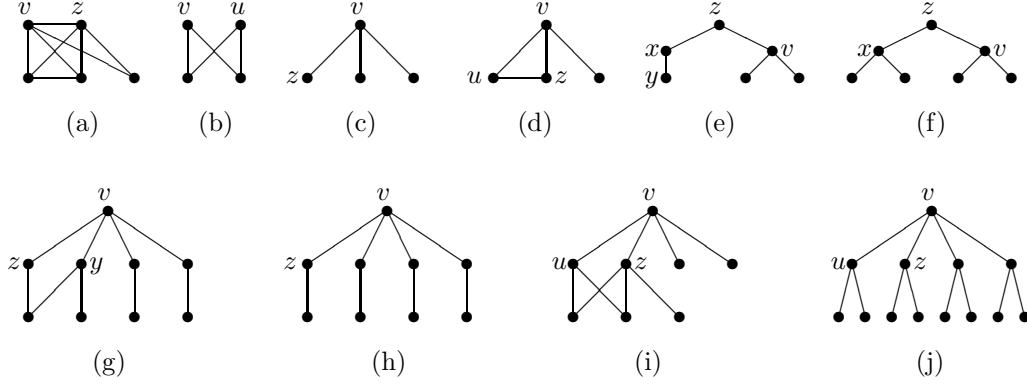


Figure 1: Worst cases of (a) True-Twin Rule 2; (b) False-Twin Rule; (c) Degree-One Vertex Rule 2; (d) Domination Rule 1; (e) Degree-Two Vertex Rule 1; (f) Degree-Two Vertex Rule 2; (g) Difference Rule 1; (h) Difference Rule 2; (i) Domination Rule 2; and (j) Maximum-Degree Rule. Edges with both endvertices in $N(v)$ are omitted.

12. **Degree-Two Vertex Rule 2.** There is a degree-two vertex z satisfying the condition that the neighbor v with larger degree between the two neighbors of z is of degree at least three. Let x be the neighbor of z other than v . Since Domination Rule is not applicable, x and v are not adjacent and $|N[u] \setminus N[v]| \geq 1$ for any $u \in N(v)$. The degree of x is at least three as Fig. 1(f). If we discard v then z becomes a degree-one vertex and hence there is an optimal solution that either selects z and discards x or selects $\{z, x\}$ and discards $N(x) \setminus \{z\}$. On the other hand, if v is selected, then there is an optimal solution that also selects one neighbor of v . Let $\mathbb{U} = \{\{v, u\} \mid u \in N(v)\}$. Hence $\beta_1(G) = \max\{1 + \beta_1(G \setminus N[z]), 2 + \beta_1(G \setminus N[\{x, z\}]), 2 + \max_{U \in \mathbb{U}} \beta_1(G \setminus N[U])\}$. The worst case happens when degree of v is three. The branching vector is $(3, 5, 5, 5, 5)$ and its branching number is 1.4336.

13. **Disconnected Rule.** If G is disconnected (let C be a component of G), return $\beta_1(G[C]) + \beta_1(G \setminus C)$.

Note that for any $d \geq 3$ there is at most one d -regular graph assigned to a node of the search tree from the root to a leaf, since every instance generated by the algorithm is an induced subgraph of the input graph of the original problem. Hence any branching rule applied to d -regular graphs, for some fixed d , can only increase the number of leaves by a multiplicative constant. Hence we may neglect the branching rules needed for d -regular graphs in the time analysis. Therefore we neglect the case when G is 3-regular. If G is not

3-regular and the maximum degree is three, then one of the above rules applies. In the following rules, we may assume that the maximum degree of the input graph is at least four.

14. **Difference Rule 1.** There is a maximum-degree vertex v of G and there exist $y, z \in N(v)$ satisfying that $|N(z) \setminus N(v)| = 1$ and $N(z) \cap N^2(v) \subseteq N(y) \cap N^2(v)$ as Fig. 1(g). The algorithm either discards v or selects v and one of vertices in $N(v) \setminus \{y\}$. Hence $\beta_1(G) = \max\{\beta_1(G \setminus \{v\}), 2 + \max_{U \in \mathbb{U}} (\beta_1(G \setminus N[U]))\}$ where $\mathbb{U} = \{\{v, p\} \mid p \in N(v) \setminus \{y\}\}$. The worst case time complexity happens when the degree of v is four. The branching vector is $(1, 6, 6, 6)$ and its branching number is 1.4570.

15. **Difference Rule 2.** There is a maximum-degree vertex v of G , a neighbor z of v , and $|N[z] \setminus N[v]| = 1$ as Fig. 1(h). Since Domination Rule is not applicable, $|N[u] \setminus N[v]| \geq 1$ for any $u \in N(v)$. In this case the algorithm either discards v or selects v and one of its neighbors. Let $\mathbb{U} = \{\{v, u\} \mid u \in N(v)\}$. Then $\beta_1(G) = \max\{\beta_1(G \setminus \{v\}), \max_{U \in \mathbb{U}} (2 + \beta_1(G \setminus N[U]))\}$. The worst case time complexity happens when the degree of v is four. The branching vector is $(1, 6, 6, 6, 6)$ and its branching number is 1.5099.

16. **Domination Rule 2.** There is a maximum-degree vertex v of G and u and z are two neighbors of v satisfying $(N(u) \setminus N[v]) \subseteq (N(z) \setminus N[v])$ as Fig. 1(i). Then the algorithm either discards v , or selects v and discards all its neighbors, or selects v and one of vertices in $N(v) \setminus \{z\}$. Let $\mathbb{U} = \{\{v, p\} \mid p \in$

$N(v) \setminus \{z\}$. Then $\beta_1(G) = \max\{\beta_1(G \setminus v), 1 + \beta_1(G \setminus N[v]), \max_{U \in \mathcal{U}} (2 + \beta_1(G \setminus N[U]))\}$. The worst case time complexity happens when the degree of v is four. The branching vector is $(1, 5, 7, 7, 7)$ and its branching number is 1.4852.

17. **Maximum-Degree Rule.** Let v be a vertex of maximum degree of G as Fig. 1(j). Since Difference Rule 2 is not applicable, $|N[u] \setminus N[v]| \geq 2$ for any $u \in N(v)$. In this case we apply **Standard Branching Rule**. With simple calculations we see that the worst case happens when the degree of v is four. The branching vector is $(1, 5, 7, 7, 7, 7)$ and its branching number is 1.5171 in this case.

Theorem 1. *The MAX-1-BDS problem can be solved in $O^*(1.5171^n)$ time.*

3 Measure and Conquer Analysis

We analyze the algorithm presented in Section 2 in two stages: (i) the maximum degree of the input graph is at least five and (ii) the maximum degree of the input graph is at most four. If the maximum degree of input graph is at least five, we can analyze the algorithm by simple analysis as Section 2. We see that the worst case happens when the Maximum-Degree Rule is applied and its branching number is $(1, 6, 8, 8, 8, 8, 8)$ and the branching number is 1.474151. For any other branching rule applied on an instance with maximum degree at least five, its branching number is less than 1.474151. For input graph with maximum degree at most four, we use measure and conquer approach.

For simplicity, we will slightly abuse the notation. Use w_i to denote the weight of a vertex of degree i , w_v to denote the weight of vertex v , $w_G = \sum_{v \in V} w_v$ to denote the weight of G where $G = (V, E)$ is the input graph. Use w for w_G if G is understood without ambiguity. We assume $w_j \geq w_i$ iff $j > i$ throughout this section. Let $\Delta w_i = w_i - w_{i-1}$ for $i > 0$. The size of the input is the weight w_G of the input graph. Suppose that we apply a branching rule b and $(t_1, t_2, \dots, t_r)$ is the corresponding branching vector. Then we obtain a recurrence

$$T(w_G) = T(w_G - t_1) + \dots + T(w_G - t_r).$$

Let α be the branching number of this rule. We see

that $O^*(\alpha^{w_G}) = O^*(\alpha^n)$ is the worst case running time of this rule since $w_G \leq n$.

All reduction rules are done in polynomial time. Hence we can focus on branching rules in the analysis. We will analyze all branching rules one by one. A branching rule may incur several recurrence relations. In Table 2, we list all recurrence relations associated with each branching rules.

Now we are ready to analyze branching rules. Consider **True-Twin Rule 2** first.

True-Twin Rule 2. The algorithm recursively solves two MAX-1-BDS problems on graphs $G_1 = G \setminus \{v, z\}$ and $G_2 = G \setminus N[v]$, respectively, in this case. The recurrence relation for this rule is $T(w) = T(w_{G_1}) + T(w_{G_2}) = T(w - t_1) + T(w - t_2)$ where $t_1 = w - w_{G_1}$ and $t_2 = w - w_{G_2}$ and the branching vector is (t_1, t_2) . For vertex u , use $\Delta_1(u)$ for the weight of u in G minus the weight of u in G_1 and $\Delta_2(u)$ for the weight of u in G minus the weight of u in G_2 . By definition,

$$t_1 = w_{d_G(v)} + w_{d_G(z)} + \sum_{u \in N(v) \setminus \{z\}} \Delta_1(u);$$

$$t_2 = \sum_{u \in N[v]} w_{d_G(u)} + \sum_{u \in N^2(v)} \Delta_2(u).$$

Since Degree-1-Edge Rule and True-Twin Rule 1 are not applicable, $d_G(v) = d_G(z) = 4$ and $|N[v] \setminus \{z\}| \geq 2$.

$$\begin{aligned} t_1 &\geq \sum_{x \in \{v, z\}} w_{d_G(x)} + \sum_{u \in N(v) \setminus \{z\}} (w_{d_G(u)} - w_{d_G(u)-2}) \geq 2w_4; \\ t_2 &\geq \sum_{u \in N[v]} w_{d_G(u)} + \sum_{u \in N^2(v)} \Delta w_{d_G(u)} \\ &\geq 3w_2 + 2w_4. \end{aligned}$$

False-Twin Rule. Suppose that the degree of v is d , $N(v) = \{u_1, u_2, \dots, u_d\}$, and u and v are false twins, i.e., $u \in N^2(v)$ and $N(u) = \{u_1, u_2, \dots, u_d\}$. Define $G_1 = G \setminus U_1$ and $G_2 = G \setminus U_2$ where $U_1 = \{v, u\}$ and $U_2 = N[\{v, u\}]$. The algorithm recursively solves two MAX-1-BDS problems on graphs G_1 and G_2 respectively. Hence

$$T(w) = T(w_{G_1}) + T(w_{G_2})$$

and

$$T(w) = T(w - (w - w_{G_1})) + T(w - (w - w_{G_2})).$$

Define $t_i = w - w_{G_i}$. Then the branching vector is (t_1, t_2) in this case. Use $\Delta_i(q)$ for the weight of

vertex q in G minus the weight of q in G_i . Then we have

$$t_i = \sum_{p \in U_i} w_{d_G(p)} + \sum_{q \in N(U_i)} \Delta_i(q).$$

Note that $2 \leq d(v) \leq 4$.

$$\begin{aligned} t_1 &\geq 2w_2 + 4 \min_{2 \leq i \leq 4} \Delta w_i, \\ t_2 &\geq 3w_2 + w_3 + \min_{1 \leq i \leq 4} \Delta w_i. \end{aligned}$$

Degree-One Vertex Rule 2. In this rule, the algorithm recursively solves two MAX-1-BDS problems on graphs $G_1 = G \setminus \{v, z\}$ and $G_2 = G \setminus N[v]$, respectively. The recurrence relation for this rule is $T(w) = T(w_{G_1}) + T(w_{G_2}) = T(w - t_1) + T(w - t_2)$ where $t_1 = w - w_{G_1}$ and $t_2 = w - w_{G_2}$ and the branching vector is (t_1, t_2) . For vertex u , use $\Delta_1(u)$ for the weight of u in G minus the weight of u in G_1 and $\Delta_2(u)$ for the weight of u in G minus the weight of u in G_2 . By definition,

$$\begin{aligned} t_1 &= w_{d_G(v)} + w_{d_G(z)} + \sum_{u \in N(v) \setminus \{z\}} \Delta_1(u); \\ t_2 &= \sum_{u \in N[v]} w_{d_G(u)} + \sum_{u \in N^2(v)} \Delta_2(u). \end{aligned}$$

Notice that $3 \leq d_G(v) \leq 4$, $d_G(z) = 1$, $d_G(u) \geq 2$ for $u \in N(v) \setminus \{z\}$, $|N^2(v)| > 1$, and $d_G(u) \geq 1$ for $u \in N^2(v)$. Hence

$$\begin{aligned} t_1 &\geq w_1 + w_3 + 2 \min_{2 \leq i \leq 4} \Delta w_i; \\ t_2 &\geq w_1 + 2w_2 + w_3 + 2 \min_{1 \leq i \leq 4} \Delta w_i. \end{aligned}$$

Domination Rule 1. Suppose the degree of v is d and $N(v) = \{u_1 = u, u_2, \dots, u_d\}$. Define $G_1 = G \setminus \{v\}$ and $G_2 = G \setminus N[\{v, u\}]$. The algorithm recursively solves two MAX-1-BDS problems on graphs G_1 and G_2 respectively. Hence

$$T(w) = T(w_{G_1}) + T(w_{G_2})$$

and

$$T(w) = T(w - (w - w_{G_1})) + T(w - (w - w_{G_2})).$$

Define $t_i = w - w_{G_i}$. Then the branching vector is (t_1, t_2) in this case. Use $\Delta_i(q)$ for the weight of vertex q in G minus the weight of q in G_i . Then we have

$$t_i = \sum_{p \in U_i} w_{d_G(p)} + \sum_{q \in N(U_i)} \Delta_i(q).$$

Since True-Twin Rule is not applicable, v has a private neighbor with respect to u . Note that $3 \leq d(v) \leq 4$.

$$\begin{aligned} t_1 &\geq w_3 + 3 \min_{2 \leq i \leq 4} \Delta w_i, \\ t_2 &\geq 3w_2 + w_3 + \min_{2 \leq i \leq 4} \Delta w_i. \end{aligned}$$

Degree-Two Vertex Rule 1. Suppose that $d(x) = 2$, the degree of v is d , and $N(v) = \{u_1 = z, u_2, \dots, u_d\}$. For $1 \leq i \leq d + 1$, define $G_i = G \setminus U_i$ where $U_1 = N[\{x, z\}]$, $U_2 = N[\{v, z\}]$, and for $2 < i \leq d + 1$, $U_i = N[\{x, y, v, u_{i-1}\}]$. The algorithm recursively solves $d + 1$ MAX-1-BDS problems on graphs $G_1, G_2, \dots, G_{d+1}$, respectively. Hence

$$T(w) = \sum_{i=1}^{d+1} T(w_{G_i}).$$

And

$$T(w) = \sum_{i=1}^{d+1} T(w - (w - w_{G_i})).$$

Define $t_i = w - w_{G_i}$. Then the branching vector is $(t_1, t_2, \dots, t_{d+1})$ in this case. Use $\Delta_i(u)$ for the weight of vertex u in G minus the weight of u in G_i . Then we have

$$t_i = \sum_{p \in U_i} w_{d_G(p)} + \sum_{q \in N(U_i)} \Delta_i(q).$$

Since Domination Rule and False-Twin Rule are not applicable, every u_j , $1 \leq j \leq d$, has a private neighbor with respect to v . Note that $3 \leq d_G(v) \leq 4$. Hence

$$\begin{aligned} t_1 &\geq 3w_2 + w_3, \\ t_2 &\geq 4w_2 + w_3, \end{aligned}$$

and for $3 \leq i \leq d + 1$,

$$t_i \geq 6w_2 + w_3.$$

Degree-Two Vertex Rule 2. Suppose that $d(x) \geq 3$, the degree of v is d , and $N(v) = \{u_1 = z, u_2, \dots, u_d\}$. For $1 \leq i \leq d + 2$, define $G_i = G \setminus U_i$ where $U_1 = N[\{z\}]$, $U_2 = N[\{x, z\}]$, $U_3 = N[\{v, z\}]$ and for $3 < i \leq d + 2$, $U_i = N[\{v, u_{i-2}\}]$. The algorithm recursively solves $d + 2$ MAX-1-BDS problems on graphs $G_1, G_2, \dots, G_{d+2}$, respectively. Hence

$$T(w) = \sum_{i=1}^{d+2} T(w_{G_i}).$$

And

$$T(w) = \sum_{i=1}^{d+2} T(w - (w - w_{G_i})).$$

Define $t_i = w - w_{G_i}$. Then the branching vector is $(t_1, t_2, \dots, t_{d+2})$ in this case. Use $\Delta_i(u)$ for the weight of vertex u in G minus the weight of u in G_i . Then we have

$$t_i = \sum_{p \in U_i} w_{d_G(p)} + \sum_{q \in N(U_i)} \Delta_i(q).$$

Since Domination Rule 1 and False-Twin Rule are not applicable, every u_j , $1 \leq j \leq d$, has a private neighbor with respect to v . Note that $3 \leq d_G(v) \leq 4$. Hence for $d_G(v) = 3$,

$$\begin{aligned} t_1 &\geq w_2 + 2w_3 + 3 \min_{2 \leq i \leq 4} \Delta w_i, \\ t_2 &\geq 3w_2 + 2w_3 + \min_{2 \leq i \leq 4} \Delta w_i, \\ t_3 &\geq 3w_2 + 2w_3 + \min_{2 \leq i \leq 4} \Delta w_i, \\ t_4 &= t_5 \geq 3w_2 + 2w_3 + \Delta w_3. \end{aligned}$$

For $d_G(v) = 4$,

$$\begin{aligned} t_1 &\geq w_2 + w_3 + w_4 + 3 \min_{2 \leq i \leq 4} \Delta w_i, \\ t_2 &\geq 3w_2 + w_3 + w_4 + \min_{2 \leq i \leq 4} \Delta w_i, \\ t_3 &\geq 4w_2 + w_3 + w_4 + \min_{2 \leq i \leq 4} \Delta w_i, \\ t_4 &= t_5 = t_6 \geq 4w_2 + w_3 + w_4 + \min_{3 \leq i \leq 4} \Delta w_i. \end{aligned}$$

Difference Rule 1. Suppose the degree of v is d and $N(v) = \{u_1, u_2, \dots, u_d = y\}$. For $1 \leq i \leq d$, define $G_i = G \setminus U_i$ where $U_1 = \{v\}$ and for $i > 1$, $U_i = N[\{v, u_{i-1}\}]$. The algorithm recursively solves d MAX-1-BDS problems on graphs $G_1, G_2, \dots, G_d$, respectively. Hence

$$T(w) = \sum_{i=1}^d T(w_{G_i})$$

and

$$T(w) = \sum_{i=1}^d T(w - (w - w_{G_i})).$$

Define $t_i = w - w_{G_i}$. Then the branching vector is $(t_1, t_2, \dots, t_d)$ in this case. Use $\Delta_i(q)$ for the weight of vertex q in G minus the weight of q in G_i . Then we have

$$t_i = \sum_{p \in U_i} w_{d_G(p)} + \sum_{q \in N(U_i)} \Delta_i(q).$$

Since Domination Rule 1 is not applicable, every u_j , $1 \leq j \leq d$, has a private neighbor with respect to v . Since False-Twin Rule and Domination Rule 1 are not applicable, $|N^2(v)| > 1$. Note that $d(v) = 4$. Let $\Delta w_{\min} = \min\{\Delta w_3, \Delta w_4\}$.

$$t_1 \geq w_4 + 4\Delta w_{\min},$$

and for $2 \leq i \leq 4$,

$$t_i \geq 5w_3 + w_4 + \Delta w_{\min}.$$

Difference Rule 2. Suppose the degree of v is d and $N(v) = \{u_1, u_2, \dots, u_d\}$. For $1 \leq i \leq d+1$, define $G_i = G \setminus U_i$ where $U_1 = \{v\}$ and for $i > 1$, $U_i = N[\{v, u_{i-1}\}]$. The algorithm recursively solves $(d+1)$ MAX-1-BDS problems on graphs $G_1, G_2, \dots, G_{d+1}$, respectively. Hence

$$T(w) = \sum_{i=1}^{d+1} T(w_{G_i})$$

and

$$T(w) = \sum_{i=1}^{d+1} T(w - (w - w_{G_i})).$$

Define $t_i = w - w_{G_i}$. Then the branching vector is $(t_1, t_2, \dots, t_{d+1})$ in this case. Use $\Delta_i(q)$ for the weight of vertex q in G minus the weight of q in G_i . Then we have

$$t_i = \sum_{p \in U_i} w_{d_G(p)} + \sum_{q \in N(U_i)} \Delta_i(q).$$

Since Domination Rule 1 is not applicable, every u_j , $1 \leq j \leq d$, has a private neighbor with respect to v . Since False-Twin Rule and Domination Rule 1 are not applicable, $|N^2(v)| > 1$. Due to Difference Rule 1 is not applicable, every u_j , $1 \leq j \leq d$ has a private neighbor with respect to any other $u_\ell \in N(v)$. We see that $|N^2(v)| \geq 4$. Note that $d(v) = 4$. Let $\Delta w_{\min} = \min\{\Delta w_3, \Delta w_4\}$.

$$t_1 \geq w_4 + 4\Delta w_{\min},$$

and for $2 \leq i \leq 5$,

$$t_i \geq 5w_3 + w_4 + 3\Delta w_{\min}.$$

Domination Rule 2. Suppose the degree of v is d and $N(v) = \{u_1, u_2, \dots, u_d = z\}$. For $1 \leq i \leq d+1$, define $G_i = G \setminus U_i$ where $U_1 = \{v\}$, $U_2 = N[v]$, and for $i > 2$, $U_i = N[\{v, u_{i-2}\}]$. The algorithm recursively solves $(d+1)$ MAX-1-BDS problems on graphs $G_1, G_2, \dots, G_{d+1}$, respectively. Hence

$$T(w) = \sum_{i=1}^{d+1} T(w_{G_i})$$

and

$$T(w) = \sum_{i=1}^{d+1} T(w - (w - w_{G_i})).$$

Define $t_i = w - w_{G_i}$. Then the branching vector is $(t_1, t_2, \dots, t_{d+1})$ in this case. Use $\Delta_i(q)$ for the weight of vertex q in G minus the weight of q in G_i . Then we have

$$t_i = \sum_{p \in U_i} w_{d_G(p)} + \sum_{q \in N(U_i)} \Delta_i(q).$$

Since Difference Rules are not applicable, every u_j , $1 \leq j \leq d$, has two private neighbors with respect to v . Note that $d(v) = 4$. Let $\Delta w_{\min} = \min\{\Delta w_3, \Delta w_4\}$. Then

$$\begin{aligned} t_1 &\geq w_4 + 4\Delta w_{\min}, \\ t_2 &\geq 4w_3 + w_4 + 8\Delta w_{\min}, \end{aligned}$$

and for $3 \leq i \leq 5$,

$$t_i \geq 6w_3 + w_4 + \Delta w_{\min}.$$

Maximum-Degree Rule. In this case the algorithm applies **Standard Branching Rule**. Suppose that the degree of v is d and $N(v) = \{u_1, u_2, \dots, u_d\}$. For $1 \leq i \leq d+2$, define $G_i = G \setminus U_i$ where $U_1 = \{v\}$, $U_2 = N[v]$, and for $i > 2$, $U_i = N[\{v, u_{i-2}\}]$. The algorithm recursively solves $d+2$ MAX-1-BDS problems on graphs $G_1, G_2, \dots, G_{d+2}$, respectively. Hence

$$T(w) = \sum_{i=1}^{d+2} T(w_{G_i})$$

and

$$T(w) = \sum_{i=1}^{d+2} T(w - (w - w_{G_i})).$$

Define $t_i = w - w_{G_i}$. Then the branching vector is $(t_1, t_2, \dots, t_{d+2})$ in this case. Use $\Delta_i(u)$ for the weight of vertex u in G minus the weight of u in G_i . Then we have

$$t_i = \sum_{p \in U_i} w_{d_G(p)} + \sum_{q \in N(U_i)} \Delta_i(q).$$

Since Difference Rules are not applicable, every u_j , $1 \leq j \leq d$, has two private neighbors with respect to v . Since Domination Rules are not applicable,

every u_j has a private neighbor in $N^2(v)$ with respect to any other $u_i \in N(v)$. Note that $d(v) = 4$. Let $\Delta w_{\min} = \min\{\Delta w_3, \Delta w_4\}$. Then

$$\begin{aligned} t_1 &\geq w_4 + 4\Delta w_{\min}, \\ t_2 &\geq 4w_3 + w_4 + 8\Delta w_{\min}, \end{aligned}$$

and for $3 \leq i \leq 6$,

$$t_i \geq 6w_3 + w_4 + 3\Delta w_{\min}.$$

In Table 2, we list branching rules and its worst case branching vectors – using measure and conquer analysis. For finding the weight vector

$$\mathbf{w} = (w_0, w_1, w_2, w_3, w_4),$$

we develop a program ² for solving the quasiconvex programming. In Table 1, we list the result returned by the program.

Table 1: The weight vector $\mathbf{w}$

$\mathbf{w}$	weights
w_0	0.000000
w_1	0.500000
w_2	0.851306
w_3	0.925947
w_4	1.000000

By substituting the weight vector $\mathbf{w}$ for the corresponding variables in the collections of recurrence functions, we can compute the branching number for each branching rule with respect to the weight vector $\mathbf{w}$. The worst case happens that when the Maximum-Degree Rule is applied and its branching number 1.483369.

Theorem 2. *The MAX-1-BDS problem can be solved in time $O^*(1.4834^n)$.*

Proof. Suppose that the maximum degree of the input graph is at least five. By computing the branching vector of each branching rules as Section 2, the worst case happens when the Maximum-Degree Rule is applied and its branching number is $(1, 6, 8, 8, 8, 8, 8)$ and the branching number is 1.474151. Due to the maximum degree condition of Maximum-Degree Rule, the algorithm ensures that at some stage all input instances are bounded degree-4. Suppose that the maximum degree of the input graph is at most four. By using measure and conquer approach, we see that the worst case happens when the Maximum-Degree Rule is applied

²<http://ufo.cs.ccu.edu.tw/qca.php>

and its branching number is $(1.296212, 5.296212, 6.777841, 6.777841, 6.777841, 6.777841)$ with respect the weight vector $\mathbf{w} = (w_0, w_1, w_2, w_3, w_4) = (0, 0.5, 0.851306, 0.925947, 1)$ and the corresponding branching number is 1.483369. Hence all branching numbers of the algorithm are bounded by 1.483369. Thus the MAX-1-BDS problem can be solved in time $O^*(1.4834^n)$. $\square$

References

- [1] B. Balasundaram, Cohesive subgroup model for graph-based text mining, *Proceedings of the 2008 IEEE Conference on Automation Science and Engineering*, pp. 989–994, 2008.
- [2] B. Balasundaram, S. Butenko, and I. V. Hicks, Clique relaxations in social network analysis: The maximum k -plex problem, *Operations Research*, **59** (2011), pp. 133–142.
- [3] B. Balasundaram, S. Chandramouli, and S. Trukhanov, Approximation algorithms for finding and partitioning unit-disk graphs into co- k -plexes, *Optimization Letters*, **4** (2010), pp. 311–320.
- [4] R. Beigel and D. Eppstein, 3-coloring in time $O(1.3289^n)$, *Journal of Algorithms*, **54** (2005), pp. 168–204.
- [5] N. Betzler, R. Brederick, R. Niedermeier, and J. Uhlmann, On bounded-degree vertex deletion parameterized by treewidth, *Discrete Applied Mathematics*, **160** (2012), pp. 53–60.
- [6] D. Binkele-Raible, *Amortized Analysis of Exponential Time- and Parameterized Algorithms: Measure & Conquer and Reference Search Trees*, PhD Thesis, Trier University, 2010.
- [7] M.-S. Chang, L.-J. Hung, and P.-C. Su, Exact and fixed-parameter algorithms for problems related to 2-plex, *Proceedings of ICSEC 2011*, pp. 203–208, 2011.
- [8] D. Eppstein, Quasiconvex analysis of backtracking algorithms, *Proceedings of SODA 2004*, pp. 781–790, 2004.
- [9] D. Eppstein, Quasiconvex analysis of multivariate recurrence equations for backtracking algorithms, *ACM Transactions on Algorithms* **2** (2006), pp. 492–509.
- [10] M. R. Fellows, J. Guo, H. Moser, and R. Niedermeier, A generalization of Nemhauser and Trotter’s local optimization theorem, *Journal of Computer and System Sciences*, **77** (2011), pp. 1141–1158.
- [11] F. V. Fomin, F. Grandoni, and D. Kratsch, Measure and conquer: domination – a case study, *Proceedings of ICALP 2005*, LNCS 3580 (2005), pp. 191–203.
- [12] F. V. Fomin, F. Grandoni, and D. Kratsch, Measure and conquer: a simple $O(2^{0.288n})$ independent set algorithm, *Proceedings of SODA 2006*, pp. 18–25, 2006.
- [13] F. V. Fomin, F. Grandoni, and D. Kratsch, A measure & conquer approach for the analysis of exact algorithms, *Journal of the ACM* **56** (2009) Article No. 25.
- [14] F. Fomin and D. Kratsch, *Exact Exponential Algorithms*, Springer, 2010.
- [15] C. Komusiewicz, F. Hüffner, H. Moser, and R. Niedermeier, Isolation concepts for efficiently enumerating dense subgraphs, *Theoretical Computer Science*, **410** (2009), pp. 3640–3654.
- [16] B. McClosky and I.V. Hicks, Combinatorial algorithms for the maximum k -plex problem, *Journal of Combinatorial Optimization*, **23** (2012) pp. 29–49.
- [17] H. Moser, R. Niedermeier, and M. Sorge, Exact combinatorial algorithms and experiments for finding maximum k -plexes, To appear in *Journal of Combinatorial Optimization* (DOI 10.1007/s10878-011-9391-5).
- [18] N. Nishmura, P. Ragde, D. M. Thilikos, Fast fixed-parameter tractable algorithms for nontrivial generalizations of vertex cover, *Discrete Applied Mathematics*, **152** (2005), pp. 229–245.
- [19] S. B. Seidman and B. L. Foster, A graph-theoretic generalization of the clique concept, *The Journal of Mathematical Sociology*, **6** (1978), pp. 139–154.
- [20] S. Trukhanov, *Novel approaches for solving large-scale optimization problems on graphs*, PhD Thesis, A&M University, Texas, 2008.

Table 2: We generate those worst cases of some specific branching rules, and then finds the weight vector $\mathbf{w}$ such that the corresponding quasiconvex programming reaches its infimum. This table lists the branching rules and the corresponding worst case branching vectors – using measure and conquer analysis. The weight vector $\mathbf{w} = (w_0, w_1, w_2, w_3, w_4) = (0, 0.5, 0.851306, 0.925947, 1)$. Substitute the weight vector to the collection of recurrence functions, we obtain that the worst branching number happens when the Maximum-Degree Rule is applied and the branching number is 1.483369.

True-Twin Rule 2 Branching Vector: (2.000000, 4.553918) Branching number: 1.250845 $T(w) = T(w - 2w_4) + T(w - 3w_2 + 2w_4)$
False-Twin Rule Branching Vector: (1.998824, 3.553918) Branching number: 1.292776 $T(w) = T(w - (2w_2 + 4 \min_{2 \leq i \leq 4} \Delta w_i)) + T(w - (3w_2 + w_3 + \min_{1 \leq i \leq 4} \Delta w_i))$
Degree One Vertex Rule 2 Branching Vector: (1.574053, 3.276665) Branching number: 1.348520 $T(w) = T(w - (w_1 + w_3 + 2 \min_{2 \leq i \leq 4} \Delta w_i)) + T(w - (w_1 + 2w_2 + w_3 + 2 \min_{1 \leq i \leq 4} \Delta w_i))$
Domination Rule 1 Branching Vector: (1.148106, 3.553918) Branching number: 1.386664 $T(w) = T(w - (w_3 + 3 \min_{2 \leq i \leq 4} w_i)) + T(w - (3w_2 + w_3 + \min_{2 \leq i \leq 4} \Delta w_i))$
Degree Two Vertex Rule 1 Branching Vector: (3.479865, 4.331171, 6.033783, 6.033783, 6.033783) Branching number: 1.381010 $T(w) = T(w - (3w_2 + w_3)) + T(w - (4w_2 + w_3)) + 3T(w - (6w_2 + w_3))$
Degree Two Vertex Rule 2 Branching Vector: (2.925359, 4.479865, 4.479865, 4.480453, 4.480453) Branching number: 1.483064 $T(w) = T(w - (w_2 + 2w_3 + 3 \min_{2 \leq i \leq 4} \Delta w_i)) + 2T(w - (3w_2 + 2w_3 + \min_{2 \leq i \leq 4} \Delta w_i)) + 2T(w - (3w_2 + 2w_3 + \Delta w_3))$
Difference Rule 1 Branching Vector: (1.296212, 5.703788, 5.703788, 5.703788) Branching number: 1.439118 $T(w) = T(w - (w_4 + 4 \min_{3 \leq i \leq 4} \Delta w_i)) + 3T(w - (5w_3 + w_4 + \min_{3 \leq i \leq 4} \Delta w_i))$
Difference Rule 2 Branching Vector: (1.296212, 5.851894, 5.851894, 5.851894, 5.851894) Branching number: 1.482352 $T(w) = T(w - (w_4 + 4 \min_{3 \leq i \leq 4} \Delta w_i)) + 4T(w - (5w_3 + w_4 + 3 \min_{3 \leq i \leq 4} \Delta w_i))$
Domination Rule 2 Branching Vector: (1.296212, 5.296212, 6.629735, 6.629735, 6.629735) Branching number: 1.455888 $T(w) = T(w - (w_4 + 4 \min_{3 \leq i \leq 4} \Delta w_i)) + T(w - (4w_3 + w_4 + 8 \min_{3 \leq i \leq 4} \Delta w_i)) + 3T(w - (6w_3 + w_4 + \min_{3 \leq i \leq 4} \Delta w_i))$
Maximum-Degree Rule Branching Vector: (1.296212, 5.296212, 6.777841, 6.777841, 6.777841) Branching number: 1.483369 $T(w) = T(w - (w_4 + 4 \min_{3 \leq i \leq 4} \Delta w_i)) + T(w - (4w_3 + w_4 + 8 \min_{3 \leq i \leq 4} \Delta w_i)) + 4T(w - (6w_3 + w_4 + 3 \min_{3 \leq i \leq 4} \Delta w_i))$